# The Capacitated Arc Routing Problem with Intermediate Facilities

**Gianpaolo Ghiani**
*Centre de recherche sur les transports, Université de Montréal, case postale 6128, succursale Centre-Ville, Montreal, Canada H3C 3J7*

**Gennaro Improta**
*Università di Napoli "Federico II," Dipartimento di Informatica e Sistemistica, via Claudio 21, 80125 Naples, Italy*

**Gilbert Laporte**
*École des Hautes Études Commerciales, 3000 chemin de la Côte-Sainte-Catherine, Montreal, Canada H3T 2A7*

**This article introduces the *Capacitated Arc Routing with Intermediate Facilities* (CARPIF), a variant of the classical *Capacitated Arc Routing Problem* (CARP) in which the vehicle may unload or replenish at intermediate facilities. Two lower bounds are developed for the CARPIF: The first is based on the *Rural Postman Problem* (RPP) and the second one uses a relaxation of an integer linear formulation of the problem. Two upper bounds are also developed, based on the solution of an RPP and of a CARP. Computational results on a set of benchmark instances confirm the quality of the proposed bounds.** © 2001 John Wiley & Sons, Inc.

**Keywords:** capacitated arc routing problem; intermediate facilities

## 1. INTRODUCTION

The purpose of this article was to develop lower bounds and heuristics for the *Capacitated Arc Routing Problem with Intermediate Facilities* (CARPIF), a variant of the classical *Capacitated Arc Routing Problem* (CARP) (see, e.g., Golden and Wong [16], Eiselt et al. [10, 11], Assad and Golden [1]). Both these problems, in their undirected version, are defined on a graph $G = (V, E)$, where $V = \{v_1, \ldots, v_n\}$ is a vertex set including a *depot* $v_1$ and $E$ is a set of edges $(v_i, v_j)(i < j)$, including a subset $R$ of *required* edges. Each edge of $R$ must be *serviced* once but can be *traversed* several times. The edges of $R$ induce $p$ connected components. Let $V_k$ be the vertex set of the $k^{th}$-connected component, where $V_1$ is the vertex set of the component containing the depot if $v_1$ is incident to an edge of $R$ and $V_1 = \{v_1\}$ otherwise. Let $V_R = \cup_{k=1}^{p} V_k$. With each edge $e = (v_i, v_j)$ are associated a traversal nonnegative *cost* or *length* $c_e = c_{ij}$ and a nonnegative *demand* $q_e = q_{ij}$. If $e \notin R$, then $q_e = 0$. Denote by $d_{ij}$ the length of a shortest chain between two vertices $v_i$ and $v_j$. In the CARP, one must determine a least-cost traversal, or tour, of all edges of $R$ such that the total demand of any chain $(v_{i_1} = v_1, v_{i_2}, \ldots, v_{i_t} = v_1)$, where $v_{i_2}, \ldots, v_{i_{t-1}} \neq v_1$, does not exceed a given vehicle capacity $Q$. In the CARPIF, the set $V$ contains a subset $I$ of *intermediate facilities* (IFs), possibly including $v_1$. The aim is to determine a least-cost tour $(v_{i_1} = v_1, v_{i_2}, \ldots, v_{i_t} = v_1)$ of all edges of $R$ such that the total demand of any chain $(v_{i_1}, \ldots, v_{i_r})$, where $v_{i_r} \in I, r \neq t, v_{i_2}, \ldots, v_{i_{r-1}} \notin I$, or of any chain $(v_{i_r}, \ldots, v_{i_s})$, where $v_{i_r}, v_{i_s} \in I, v_{i_{r+1}}, \ldots, v_{i_{s-1}} \notin I$, does not exceed $Q$. In other words, starting from the depot, the vehicle traverses edges of $E$, collects demands on required edges, and visits IFs to unload. The total demand accumulated between the depot and the first IF or between two successive IFs may never exceed $Q$. If the last IF on the tour is not the depot, then the final chain between that facility and the depot does not have any positive demand and is therefore unconstrained (see Fig. 1). This article can be viewed as a first step toward solving more general problems involving several vehicles, as is often the case in practice. Our results can be used to generate good solutions to multivehicle problems by first assigning required edges to individual vehicles and then working on each route separately.

A different but conceptually equivalent version of the CARPIF is where the vehicle makes deliveries instead of collections. Then, it must replenish at IFs in order to

Received April 1999; accepted October 2000
*Correspondence to*: G. Laporte
Contract Grant Sponsor: Natural Sciences and Research Council of Canada (NSERC); contract grant number: OPG0039682
Contract grant sponsor: Consiglio Nazionale delle Ricerche of Italy (CNR)
© 2001 John Wiley & Sons, Inc.

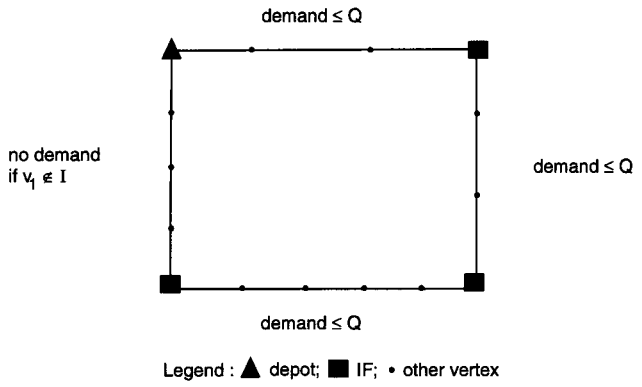



FIG. 1. Feasible CARPIF solution (the tour is in the clockwise direction starting from the depot).

satisfy the demand until it reaches the next facility or the depot. Applications of the first version of the CARPIF arise, for example, in garbage collection where vehicles make intermediate visits at dump sites or incinerators. Such a system was described by Wirasinghe and Waters [23] for the city of Calgary in Canada. Instances of the second version are encountered in road gritting where IFs are conveniently located sand or chemical boxes (see, e.g., Lemieux and Campagna [21] and Eglese and Li [9]). What follows applies to the first version of the CARPIF (with demand deliveries). Adaptations of our results to the second version are straightforward.

The CARPIF is NP-hard since it reduces to the CARP, another NP-hard problem [15], whenever $I = \{v_1\}$. In addition, if $\{v_1\} \subset I$, then the CARPIF is a relaxation of the CARP since any CARP solution is feasible for the CARPIF. If $I = \{v_r\}$ but $r \neq 1$, then the corresponding CARPIF can easily be viewed as a CARP with a depot located at $v_r$ provided that an artificial required edge $(v_1, v_r)$ is introduced with $q_{1r} = 0$ and $c_{1r} = d_{1r}$. The following dominance check and reduction procedure can sometimes be used to remove some dominated IFs: For each $v_i, v_j \in V_R$, let $f^*_{ij} = \arg\min_{f \in I}\{d_{if} + d_{fj}\}$ and reset $I := \cup_{v_i, v_j \in V_R}\{f^*_{ij}\}$. In other words, if $f \in I$ is dominated by some other IF for every pair of vertices $\{v_i, v_j\}$, then there always exists an optimal solution not containing $f$, and this facility can therefore be discarded.

The remainder of this article is organized as follows: In Section 2, we develop a lower-bounding procedure for the CARPIF based on the formulation of one of its relaxations. Two heuristics are presented in Section 3. Computational results are provided in Section 4, followed by conclusions in Section 5.

## 2. LOWER BOUNDS

In this section, we develop two lower bounds for the CARPIF. The first is based on the *Rural Postman Problem* (RPP), a natural relaxation of the CARPIF. The second uses an integer linear formulation of the problem with deadheading variables.

### 2.1. The RPP Lower Bound

First, observe that the CARP solution does not provide a valid lower bound on the value of the corresponding CARPIF. To see this, consider the graph depicted in Figure 2, where $V = I = \{v_1, v_2, v_3, v_4\}, c_{1,2} = c_{2,3} = c_{3,4} = c_{1,4} = 1, q_{1,2} = q_{2,3} = q_{3,4} = q_{1,4} = 1$, and $Q = 1$. Denote by $z^*_{CARPIF}$ and $z^*_{CARP}$ the optimal CARPIF and CARP solution values, respectively. Then, $z^*_{CARPIF} = 4$, while $z^*_{CARP} = 12$.

However, the RPP (which can be defined as a CARP with $Q = \infty$) provides a natural lower bound for the CARPIF. If $z^*_{RPP}$ is the optimal RPP solution value associated with a given graph, then $z^*_{RPP} \leqq z^*_{CARPIF}$. Since the RPP is itself NP-hard, it may not be practical solve this problem optimally. Instead, one can use an easy-to-compute lower bound $\underline{z}_{RPP}$ on $z^*_{RPP}$ which can be generated by means of a branch-and-cut procedure. Thus, using an integer linear programming formulation and additional valid inequalities, Ghiani and Laporte [14] were able to obtain $\underline{z}_{RPP}$ values at the root of the branch-and-cut tree, varying between 0.987 and 0.999 of $z^*_{RPP}$, on randomly generated graphs containing between 50 and 350 vertices, within very short computing times. While it would seem natural to use an RPP lower bound, this bound can sometimes be quite weak since it ignores capacity constraints and the location of IFs. For this reason, we have developed an alternative bound.

### 2.2. A Lower Bound Based on Deadheading Variables

As for the CARP, it is rather difficult to come up with a workable integer linear programming formulation of the CARPIF that does not contain too many variables and constraints. However, a good relaxation can be formulated in terms of so-called *deadheading variables*. Let $z_e$ be an integer variable representing the number of times edge $e$ is deadheaded: If $e \in R$, then $z_e$ is equal to the number of times $e$ is traversed, minus 1; if $e \in E \backslash R$, then $z_e$ is the number of traversals of $e$. Using these variables, we can formulate a relaxation of the CARPIF in the spirit of the Belenguer and Benavent [3, 4] formulation for the CARP. A lower bound for the CARPIF is then obtained by optimally solving this relaxation to

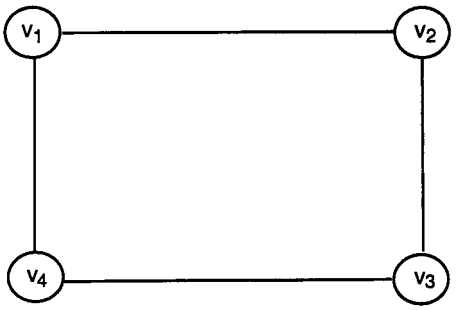



FIG. 2. The CARP does not yield a valid lower bound for the CARPIF.

optimality by branch-and-cut. This does not, however, always provide a feasible CARPIF solution.

Given a vertex set $S \subseteq V$, let $E(S)$ be the edge set induced by $S$, that is, $E(S) = \{(v_i, v_j) \in E : v_i, v_j \in S\}$, and let $\delta(S) = \{(v_i, v_j) \in E : v_i \in S, v_j \in V\backslash S$ or $v_i \in V\backslash S, v_j \in S\}$. Also, let $E_R(S) = E(S) \cap R$ and $\delta_R(S) = \delta(S) \cap R$. If $E' \subseteq E$, then $z(E') = \sum_{e \in E'} z_e$. Given $S \subseteq V$, let $D(S) = \sum_{e \in E_R(S) \cup \delta_R(S)} q_e$ and let $h(S)$ be a lower bound on the number of stops at an IF required to service all edges incident to the vertices of $S$. In our implementation, we use $h(S) = \lceil D(S)/Q \rceil$ as is commonly done in the Vehicle Routing Problem. A vertex set $S \subset V$ is said to be $R$-odd ($R$-even) if an odd (even) number of required edges are incident to $S$. Finally, a *backtree* $T = (V_T, E_T)$ of $G$ is a tree whose edge set is $E_T = E_1 \cup \ldots \cup E_{|I|}$, where $E_f$ is the edge set on a shortest chain between $f \in I$ and $v_1$. We then propose the following CARPIF relaxation (CR):

$$(CR) \text{ minimize } \sum_{e \in E} c_e z_e \tag{1}$$

subject to

$$z(\delta(S)) \geqq 2 (S = \cup_{k \in P} V_i, \varnothing \neq P \subseteq \{2, \ldots, p\}) \tag{2}$$

$$z(\delta(S)) \geqq 2h(S) - |\delta_R(S)| \qquad (S \subseteq V\backslash I) \tag{3}$$

$$z(\delta(S)) \geqq 1 \qquad (S \subset V, S \text{ is } R - odd) \tag{4}$$

$$z(\delta(S) \cap E_T) \geqq 1 \qquad (S \subset V, v_1 \in S, S \cap I = \varnothing) \tag{5}$$

$$z(\delta(S)) \geqq 2 \quad (S \subset V, v_1 \in S, S \cap I = \varnothing, S \text{ is } R - \text{even}) \tag{6}$$

$$z_e \geqq 0 \qquad (e \in E). \tag{7}$$

In (CR) *connectivity constraints* (2), stipulate that all connected components are connected by at least two edges to their complement. Constraints (3) are referred to as the *CARPIF capacity constraints* and are based on the fact that at least $h(S)$ vehicle trips are required to service $E_R(S) \cup \delta_R(S)$ and each trip involves one journey in each direction between $S$ and $I$. The constraint is valid since the number of vehicle trips between $S$ and $V\backslash S$ is equal to $z(\delta(S)) + |\delta_R(S)|$. These constraints are formally equivalent to the capacity constraints used by Belenguer and Benavent [4] for the CARP, with the exception that constraints (3) hold for $S \subseteq V\backslash I$, whereas the CARP constraints hold for $S \subseteq V\backslash\{v_1\}$. Observe that if $S \cap I = \varnothing$ connectivity constraints (2) are dominated by (3), since if $S = \cup_{k \in P} V_i$ and $\varnothing \neq P \subseteq \{2, \ldots, p\}$, then $|\delta_R(S)| = 0$ and $h(S) \geqq 1$. However, constraints (2) are still useful since (3) does not apply if $S \cap I \neq \varnothing$, whereas (2) does. Constraints (4) are the standard $R$-odd inequalities used in the RPP (Corberán and Sanchis [7]). They state that if $S$ is $R$-odd then the CARPIF solution must contain at least one deadhead edge between $S$ and $V\backslash S$. There is no dominance relationship between constraints (3) and (4). Constraints (5) are called *backtree inequalities*. They apply if $v_1 \notin I$. These cuts are valid since there exists an optimal CARPIF solution containing at least one deadheaded edge between $S$ and its complement, on a shortest chain between the last visited IF and $v_1$, whenever $v_1 \in S$. Finally, constraints (6) are called *twin backtree inequalities*. They apply whenever a set $S$ is $R$-even and contains $v_1$ but no IF. Then, there exists an optimal CARPIF solution containing in addition to an edge of the backtree at least another deadheaded edge between $S$ and its complement.

In (CR), the objective function as well as constraints (2), (3), (4), and (7) are those of Belenguer and Benavent [4], while (5) and (6) are new and particular to the CARPIF. A lower bound $z_{\text{CARPIF}}$ on $z^*_{\text{CARPIF}}$ is obtained by solving an initial relaxation of (CR) and generating relaxed constraints in a dynamic fashion, as they are found to be violated. The initial relaxation is made up of (1), (7), and subsets of (2), (3), and (4). Initially, one constraint of type (2) is generated for every connected component. For the generation of constraints (3), we applied the following procedure:

STEP 1. Set $U := I$.
STEP 2. Set $S := V\backslash U$. Generate constraint (3) applied to $S$ if its right-hand side is positive.
STEP 3. Include in $U$ all vertices adjacent to at least one vertex of $U$. If $U = V$, stop. Otherwise, go to Step 2.

One constraint (4) was also generated for each $R$-odd vertex. After solving the initial relaxation, we repeatedly generated all identified violated constraints (2)–(6) until no violation could be detected. For this, we used the following separation procedures:

**Constraints (2).** Although the separation problem is solvable in $O(|V|^3)$ time [20], as for the RPP, we used the heuristic procedure proposed by Fischetti et al. [12]. A maximum spanning tree is built on an auxiliary graph $\bar{G}$, where each connected component $h$ of $R$ is represented by a vertex $v'_h$ in $\bar{G}$, and edges $e' = (v'_k, v'_l)$ have a cost equal to the sum of variable values $z_e$ corresponding to edges $(v_i, v_j)$ such that $v_i \in V_k$ and $v_j \in V_l$. At any stage of the construction of this tree, let $S$ be the set of connected components of $R$ corresponding to vertices of the partial tree. If $S$ yields a violation of (2), the corresponding constraint is generated. Once the spanning tree is complete, another check for violated connectivity constraints is made by removing, in turn, each edge of the tree.

**Constraints (3).** We separated (3) using $h(S) = D(S)/Q$ and applying a modification of the procedure proposed by Belenguer and Benavent [4] for the CARP to take into account the fact that the CARPIF contains IFs in addition to the depot. Thus, instead of solving maximum-flow/minimum-cut problems involving the depot and some other vertices, we solve such problems on an auxiliary graph $\tilde{G}$ where a dummy vertex $v_0$ is connected to

every IF and another dummy vertex $v_{n+1}$ is connected to every $v_i \in V_R$. The capacity $u_e$ of every edge $e$ of $\tilde{G}$ is defined as follows:

$$u_e = \begin{cases} z_e & if \ \ e \in E\backslash R, \\ z_e + 1 - q_e/Q & if \ \ e \in R, \\ D(\delta_R(\{v_i\}))/Q & if \ \ e = (v_i, v_{n+1}), v_i \in V_R, \\ \infty & if \ \ e = (v_0, v_j), v_j \in I. \end{cases}$$

Using reasoning similar to that of Belenguer and Benavent [4], we observe that (1) a minimum cut separating $v_0$ and $v_{n+1}$ corresponds to a cut edgeset $(\bar{V}, V\backslash\bar{V})$ such that $v_0 \in \bar{V}, v_{n+1} \in V\backslash\bar{V}, I \subseteq \bar{V}$, and (2) given $S \subseteq V\backslash I$, the capacity of the cut $\delta(S \cup \{v_{n+1}\})$, minus the constant $2D(S)/Q$, is equal to the slack of constraint (3) for vertex set $S$. It follows that if the maximum flow from $v_0$ to $v_{n+1}$ minus $2D(S)/Q$ is negative then constraint (3) corresponding to a minimum cut $\delta(S^*)$ that separates $v_0$ and $v_{n+1}$ is violated by the current solution. Otherwise, no constraint (3) is violated.

**Constraints (4).** Here, we applied the $O(|V|^4)$ exact procedure of Padberg and Rao [22].

**Constraints (5).** To identify violated backtree inequalities, we developed the following procedure:

STEP 1. Define an auxiliary graph $G' = (V', E')$, where $V' = V_T \cup \{v_0\}$ and $E' = E_T \cup \{(v_0, v_f) : v_f \in I\}$. Assign to $e = (v_0, v_f)$ a capacity $u_e = \infty$ and to every edge $e$ of $E_T$ a capacity $u_e = z_e$.
STEP 2. Determine in $G'$ a minimum cut $(S^*, V'\backslash S^*)$ separating $v_1$ from $v_0$ (in practice, separating $v_1$ from the vertices of $I$).
STEP 3. If the capacity of this cut is less than 1, constraint (5) corresponding to $S = S^*$ is violated. Otherwise, no constraint (5) is violated.

**Constraints (6).** We construct an auxiliary graph $G' = (V', E')$, where $V' = V \cup \{v_0\}, E' = E \cup \{(v_0, v_f) : v_f \in I\}$. Assign to $e = (v_0, v_f)$ a capacity $u_e = \infty$ and to every edge $e$ of $E$ a capacity $u_e = z_e$. Then, compute a minimum *odd cut* $(S^*, V'\backslash S^*)$ on $G'$ separating $v_0$ and $v_1$. Recall that a odd cut is such that $S^*$ contains an odd number of odd vertices. For our purpose, the parity of every vertex of $V'$ is determined as follows: $v_0$ is odd, $v_1$ is odd if it is $R$-even and even if it is $R$-odd, and every vertex of $V'\backslash\{v_0, v_1\}$ is even if it is $R$-even and odd if it is $R$-odd. The problem of determining a minimum odd cut $(S^*, V'\backslash S^*)$ in a graph $G' = (V', E')$ can be solved in $O(|V'|^4)$ time (Padberg and Rao [22]). However, no polynomial time algorithm is known for this problem if each of $S^*$ and $V'\backslash S^*$ must contain a specified vertex, as is the case in the CARPIF (Grötschel et al. [16]). We have therefore devised the following heuristic:

STEP 1. Determine a minimum capacity cut $(S^*, V'\backslash S^*)$ on $G'$. If $S^*$ is odd and corresponds to a violated constraint (6), generate this constraint and go to Step 2. If $S^*$ is even, go to Step 3.
STEP 2. Consider, in turn, each even vertex $v \in S^*\backslash\{v_1\}$, set $S^* := S^*\backslash\{v\}$, and compute the slack of constraint (6) with $S := S^*$. Also, consider, in turn, each even vertex $v \in V\backslash S^*$, set $S^* := S^* \cup \{v\}$, and compute the slack of constraint (6). If no violation of (6) has been identified, stop. Otherwise, generate every violated constraint and stop.
STEP 3. Consider, in turn, each odd vertex $v \in S^*\backslash\{v_0, v_1\}$, set $S^* := S^*\backslash\{v\}$, and compute the slack of constraint (6). Also, consider, in turn, each odd vertex $v \in V\backslash S^*$, set $S^* := S^* \cup \{v\}$, and compute the slack of constraint (6). If no violation of (6) has been identified, stop. Otherwise, generate every violated constraint and stop.

## 3. HEURISTICS

We have developed two heuristics for the CARP. The first computes an RPP solution on $G$ and derives from it a feasible CARPIF solution by optimally introducing IFs by means of a shortest path algorithm. The second applies a CARP heuristic to a transformed graph, makes the solution feasible for the CARPIF, and finally applies an improvement heuristic.

### 3.1. RPP-based Heuristic

In the RPP-based heuristic, edge demands are first ignored and an RPP feasible solution is determined by means of Frederickson's heuristic [13], which first constructs a shortest tree spanning all components of required edges and then computes an optimal matching of odd-degree vertices. In a second stage, demands are reintroduced and IFs are optimally inserted for each orientation of the RPP solution by solving a shortest path problem on an auxiliary directed graph, in the spirit of Beasley's [2] route-first/cluster-second heuristic for the capacitated *Vehicle Routing Problem* and of Jansen's [19] heuristic for the CARP. More formally, the heuristic can be described as follows:

STEP 1. Apply Frederickson's RPP heuristic on $G$.
STEP 2. For each orientation of the RPP solution, construct an auxiliary directed graph $H = (W, A)$ as follows: Let $e_r = (v_{i_r}, v_{j_r})$ be the $r^{th}$-serviced edge of the solution starting from the depot. The vertex set $W$ is made up of a source $s$ and a sink $t$ and a vertex $w_{hk}$ corresponding to each sequence $e_h, \ldots, e_k$ of serviced edges, provided that $\sum_{r=h}^{k} q_{i_r j_r} \leqq Q$. The arc set $A$ contains an arc of cost $d_{1i_1}$ from $s$ to every vertex $w_{1,k}$, an arc of cost $\min_{f \in I}\{d_{j_{|R|}f} + d_{f1}\}$ from every vertex $w_{h,|R|}$ to $t$, and an arc of cost $\min_{f \in I}\{d_{j_k f} + d_{f i_{k+1}}\}$ from every vertex $w_{hk}$ to every vertex $w_{k+1,l}$. A feasible CARPIF solution is then obtained by determining a least-cost path from $s$ to

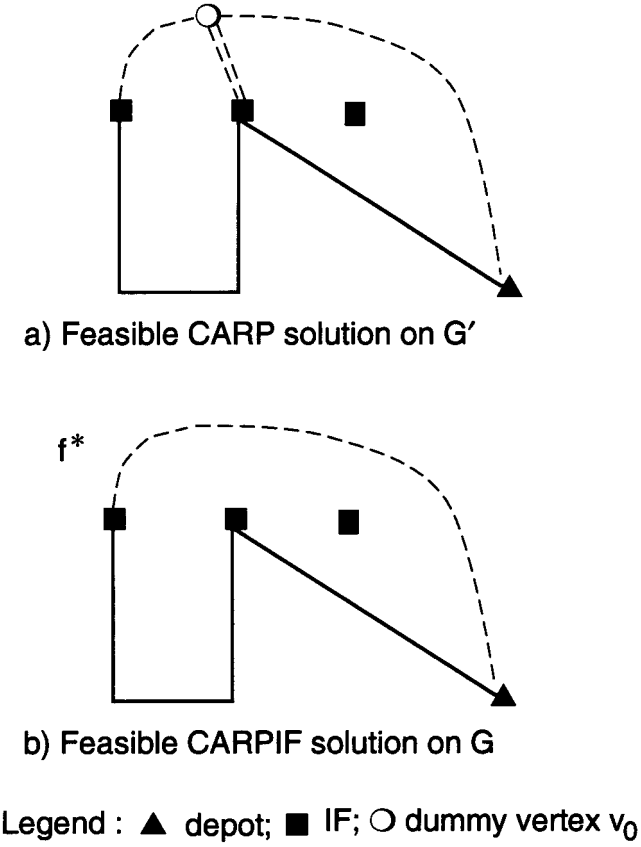



FIG. 3. Feasible CARPIF solution yielded by the CARP-based heuristic.

$t$ on $W$, which implicitly provides feasible sequences of serviced edges and IFs. The best of the two solutions corresponding to the two orientations of the RPP solution is selected.

### 3.2. *CARP-based Heuristic*

We have also devised a second heuristic in which a CARP is first solved on a modified graph $G'$. If the solution is feasible for the CARPIF, the process ends. Otherwise, a feasible CARPIF solution is obtained by solving an RPP on an auxiliary multigraph $H$, and an improvement is sought in a third and final step. These three steps are now described:

STEP 1. Define $G' = (V', E')$, where $V' = V \cup \{v_0\}$ and $E' = E \cup \{(v_0, v_1)\} \cup \{(v_0, f) : f \in I\}$. Let $R' = R \cup \{(v_0, v_1)\}$. Assign a zero demand to all edges of $E' \backslash E$, a zero cost to all $(v_0, f)$ edges ($f \in I$), and define $c_{0,1} = \min_{f \in I}\{d_{1f}\}$. Solve a CARP on $G'$ with required edge set $R'$ and a depot located at $v_0$. Delete $v_0$ and all edges of $E' \backslash E$ and replace edge $(v_0, v_1)$ by a shortest chain between $v_1$ and $v_{f^*}$, where $f^* = \arg\min_{f \in I}\{d_{1f}\}$. If the graph resulting from the CARP solution and the subsequent edge deletions is Eulerian, it then constitutes a feasible CARPIF solution since it can be decomposed into a connected set of routes of demand not exceeding $Q$, all starting and ending at an IF, and one of these routes contains a zero-demand chain between $v_1$ and $f^* \in I$. The procedure then terminates. In our implementation, the CARP is solved heuristically using the CARPET algorithm of Hertz et al. [17]. If the graph is not Eulerian, it corresponds to an infeasible CARPIF solution. The algorithm proceeds to Step 2 in order to make the resulting graph Eulerian. The feasible and infeasible cases are depicted in Figures 3 and 4.

STEP 2. Define an auxiliary undirected multigraph $H = (W, F)$. The vertex set $W$ is the subset of IFs selected by the CARP algorithm in Step 1. The edge set $F$ contains one required edge for each IF-to-IF chain in the CARP solution translated on $G$, as well as one nonrequired edge corresponding to each shortest chain on $G$ between any pair of two different vertices of $W$. Assign to each required edge the value of its corresponding cost in $G$ and to each nonrequired edge a cost equal to the length of the corresponding shortest chain in $G$. Solve the RPP on $H$ and translate the solution into a CARPIF solution on $G$ (the above construction process guarantees that it is always feasible). In our implementation, the RPP is solved heuristically by first applying Frederickson's [13] heuristic and then the SHORTEN procedure developed by Hertz et al. [18] for the RPP.

STEP 3. In the RPP solution on $G$, consider, in turn, all chains $C_k$ of the form $(f_1, \ldots, f_2)$, where $f_1$ and $f_2 \in I$,

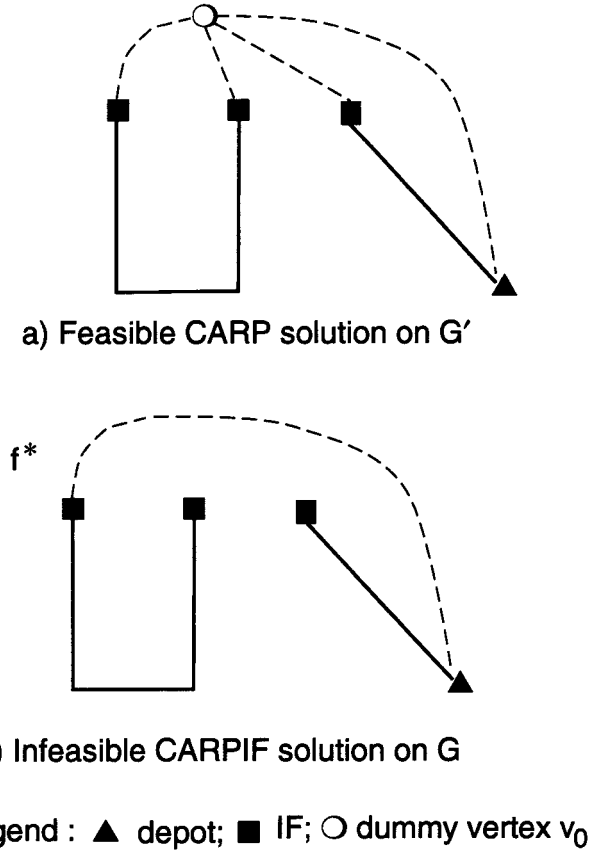



FIG. 4. Infeasible CARPIF solution yielded by the CARP-based heuristic.

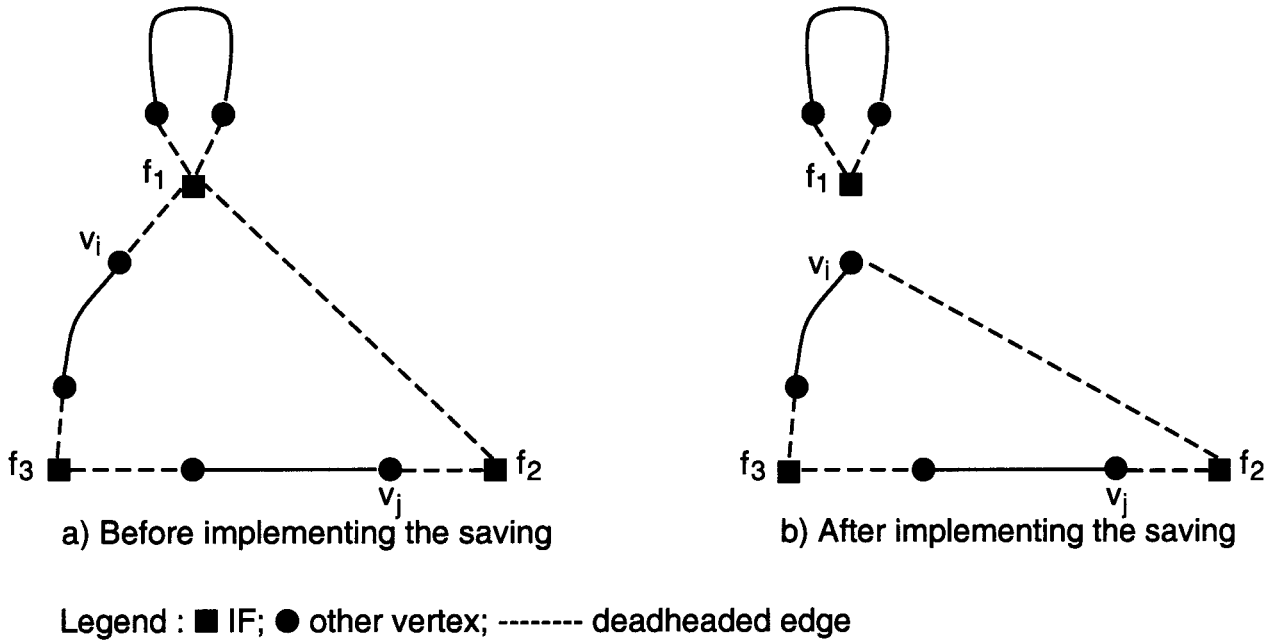



FIG. 5. Saving using the chain $(v_i, f_1, f_2)$ and yielding an infeasible CARPIF solution in Step 3 of the CARP-based heuristic.

no intermediate vertex is an IF, and all edges of the chain are deadheaded. For each chain $C_k$, identify all maximal chains $C_{kl}$ (made up exclusively of deadheaded edges) of the form $(v_i, \ldots, f_1, \ldots, f_2, \ldots, v_j)$ and determine whether a saving $s_{kl}$ can be obtained by using a shortest path between $v_i$ and $f_2$ or between $f_1$ and $v_j$. Order all savings $s_{kl}$ in nondecreasing order. Starting from the top of the list, implement, in turn, any saving that does not yield an infeasible solution (see Fig. 5 for such an example). If a saving $s_{kl}$ is implemented, then delete from the list all savings $s_{kl'}$, where $l' \neq l$.

## 4. COMPUTATIONAL RESULTS

The lower- and upper-bounding procedures just described were coded in C and run on a Sun Ultra-10 station, 300 MHz, 128 Mbytes. The linear programs used for the second lower bound were run on CPLEX [7], suitably adapted to encompass the separation procedures and the constraint generation mechanism. The CARPET algorithm used for the second heuristic was translated into C from the Pascal program of Hertz et al. [17].

The procedures were tested on three sets of benchmark instances adapted to the CARPIF. The first set

TABLE 1. Lower-bound results for the DeArmon instances.

| Instance no. | $\|V\|$ | $\|E\|$ | LB1 | LB2 | R-ODD | CAPACITY | BACKTREE | TWIN BACKTREE | NODES |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 12 | 22 | 294 | 308 | 9 | 3 | 0 | 0 | 1 |
| 2 | 12 | 26 | 315 | 335 | 6 | 2 | 1 | 7 | 1 |
| 3 | 12 | 22 | 259 | 275 | 7 | 2 | 1 | 5 | 1 |
| 4 | 11 | 19 | 266 | 296 | 8 | 2 | 0 | 0 | 1 |
| 5 | 13 | 26 | 346 | 427 | 10 | 2 | 1 | 7 | 1 |
| 6 | 12 | 22 | 279 | 359 | 6 | 2 | 2 | 9 | 1 |
| 7 | 12 | 22 | 304 | 342 | 11 | 2 | 0 | 0 | 1 |
| 10 | 27 | 46 | 250 | 404 | 24 | 9 | 1 | 21 | 1 |
| 11 | 27 | 51 | 247 | 323 | 19 | 9 | 1 | 19 | 1 |
| 12 | 12 | 25 | 275 | 294 | 8 | 2 | 2 | 11 | 1 |
| 13 | 22 | 45 | 387 | 418 | 27 | 2 | 1 | 22 | 1 |
| 14 | 13 | 23 | 384 | 561 | 8 | 6 | 3 | 13 | 1 |
| 15 | 10 | 28 | 520 | 576 | 6 | 2 | 1 | 6 | 1 |
| 16 | 7 | 21 | 96 | 110 | 0 | 1 | 1 | 5 | 1 |
| 17 | 7 | 21 | 56 | 58 | 0 | 1 | 0 | 0 | 1 |
| 18 | 8 | 28 | 125 | 127 | 8 | 1 | 0 | 0 | 1 |
| 19 | 8 | 28 | 91 | 93 | 8 | 1 | 0 | 0 | 1 |
| 20 | 9 | 36 | 158 | 168 | 0 | 1 | 1 | 7 | 1 |
| 21 | 10 | 11 | 55 | 67 | 5 | 1 | 1 | 5 | 1 |
| 22 | 11 | 22 | 121 | 145 | 7 | 2 | 1 | 3 | 1 |
| 23 | 11 | 33 | 154 | 160 | 4 | 1 | 0 | 0 | 1 |
| 24 | 11 | 44 | 196 | 205 | 6 | 2 | 1 | 7 | 1 |
| 25 | 11 | 55 | 223 | 235 | 0 | 2 | 0 | 0 | 1 |

comprises the 25 instances of DeArmon [8] from which instances 8 and 9 were removed because they contained inconsistencies. Also, in instance 21, the isolated vertex 11 was removed. The size of these instances ranges from 7 to 27 vertices and from 11 to 55 edges, all of which are required. An IF was located at vertex $|V|$. The second set is made up of 34 instances supplied by Benavent et al. [5]. In these, all edges are again required. These instances range in size from 24 to 50 vertices and from 34 to 97 edges. Two IFs were located at vertices $\lfloor |V|/2 \rfloor$ and $2\lfloor |V|/2 \rfloor$. The third set contains 30 instances and was obtained as follows: Using the same procedure as that of Hertz et al. [17], we first generated 10 instances, with $|V| = 40$, edge density in the range [0.4, 0.6], and a proportion of required edges in the range [0.4, 0.6]. Three instances were then produced in each case by selecting one, two, or three IFs at vertices $|V|$, $\lfloor |V|/2 \rfloor$, and $\lfloor |V|/3 \rfloor$. These instances contain between 351 and 445 edges and between 161 and 263 required edges. Computational results on the behavior of the lower and upper bounds are presented in Tables 1–6. The column headings are as follows:

| | |
|---|---|
| $\|V\|$ | number of vertices |
| $\|E\|$ | number of edges (all required) |
| LB1 | RPP lower-bound value |
| LB2 | deadheading variables' lower-bound value |
| R-ODD | number of *R*-odd inequalities generated for LB2 |
| CAPACITY | number of CARPIF capacity constraints generated for LB2 |
| BACKTREE | number of backtree inequalities generated for LB2 |
| TWIN BACKTREE | number of twin backtree inequalities generated for LB2 |
| NODES | number of branch-and-cut nodes for LB2 |
| UB1 | solution value produced by the RPP-based heuristic |
| SECONDS UB1 | computing time in seconds for UB1 |
| UB2 | solution value produced by the CARP-based heuristic |
| SECONDS UB2 | computing time in seconds for UB1 |
| DEVIATION % | percentage deviation of the best upper-bound value over the best lower-bound value, that is, $100[(\min\{UB1, UB2\}/\max\{LB1, LB2\}) - 1]$ |

The computational results presented in Tables 1 and 3 indicate that LB2 is always superior to LB1. Since all graphs induced by the required edges (all edges) are connected, no connectivity constraint is ever generated for the computation of LB2. Otherwise, the most useful constraints are the *R*-odd inequalities and the twin backtree inequalities. The computation times required for LB1 or LB2 are always around 1 second for the DeArmon instances and between 0 and 2.5 seconds for the Benavent and for the Hertz–Laporte–Mittaz instances.

TABLE 2. Upper-bound results for the DeArmon instances.

| Instance no. | $\|V\|$ | $\|E\|$ | UB1 | SECONDS UB1 | UB2 | SECONDS UB2 | DEVIATION % |
|---|---|---|---|---|---|---|---|
| 1 | 12 | 22 | 336 | 1 | 308 | 203 | 0.00 |
| 2 | 12 | 26 | 369 | 1 | 335 | 485 | 0.00 |
| 3 | 12 | 22 | 295 | 1 | 275 | 264 | 0.00 |
| 4 | 11 | 19 | 341 | 1 | 296 | 185 | 0.00 |
| 5 | 13 | 26 | 480 | 2 | 428 | 497 | 0.23 |
| 6 | 12 | 22 | 399 | 1 | 359 | 565 | 0.00 |
| 7 | 12 | 22 | 391 | 1 | 342 | 202 | 0.00 |
| 10 | 27 | 46 | 468 | 7 | 416 | 3184 | 2.97 |
| 11 | 27 | 51 | 401 | 9 | 337 | 337 | 4.33 |
| 12 | 12 | 25 | 354 | 2 | 294 | 332 | 0.00 |
| 13 | 22 | 45 | 517 | 17 | 430 | 1082 | 2.87 |
| 14 | 13 | 23 | 640 | 1 | 593 | 590 | 5.70 |
| 15 | 10 | 28 | 638 | 1 | 594 | 829 | 3.13 |
| 16 | 7 | 21 | 118 | 1 | 110 | 300 | 0.00 |
| 17 | 7 | 21 | 60 | 1 | 58 | 181 | 0.00 |
| 18 | 8 | 28 | 134 | 1 | 129 | 530 | 1.57 |
| 19 | 8 | 28 | 95 | 3 | 93 | 345 | 0.00 |
| 20 | 9 | 36 | 176 | 5 | 168 | 435 | 0.00 |
| 21 | 10 | 11 | 69 | 1 | 67 | 76 | 0.00 |
| 22 | 11 | 22 | 165 | 1 | 145 | 324 | 0.00 |
| 23 | 11 | 33 | 172 | 3 | 164 | 701 | 2.50 |
| 24 | 11 | 44 | 222 | 7 | 209 | 1657 | 1.95 |
| 25 | 11 | 55 | 251 | 16 | 243 | 3430 | 3.40 |

TABLE 3. Lower-bound results for the Benavent instances.

| Instance no. | $\|V\|$ | $\|E\|$ | LB1 | LB2 | R-ODD | CAPACITY | BACKTREE | TWIN BACKTREE | NODES |
|---|---|---|---|---|---|---|---|---|---|
| 1A | 24 | 39 | 173 | 183 | 22 | 0 | 3 | 36 | 1 |
| 1B | 24 | 39 | 173 | 183 | 22 | 0 | 3 | 36 | 1 |
| 1C | 24 | 39 | 173 | 205 | 25 | 18 | 3 | 27 | 7 |
| 2A | 24 | 34 | 217 | 247 | 16 | 0 | 3 | 46 | 1 |
| 2B | 24 | 34 | 217 | 247 | 16 | 0 | 3 | 46 | 1 |
| 2C | 24 | 34 | 217 | 312 | 18 | 11 | 3 | 32 | 1 |
| 3A | 24 | 35 | 77 | 85 | 22 | 0 | 6 | 49 | 1 |
| 3B | 24 | 35 | 77 | 85 | 22 | 0 | 6 | 49 | 1 |
| 3C | 24 | 35 | 77 | 91 | 20 | 12 | 4 | 40 | 1 |
| 4A | 41 | 69 | 388 | 402 | 33 | 0 | 4 | 81 | 1 |
| 4B | 41 | 69 | 388 | 402 | 33 | 1 | 4 | 81 | 1 |
| 4C | 41 | 69 | 388 | 402 | 27 | 1 | 4 | 76 | 1 |
| 4D | 41 | 69 | 388 | 428 | 28 | 9 | 6 | 4 | 1 |
| 5A | 34 | 65 | 415 | 432 | 36 | 0 | 11 | 116 | 1 |
| 5B | 34 | 65 | 415 | 432 | 37 | 1 | 11 | 105 | 3 |
| 5C | 34 | 65 | 415 | 434 | 33 | 2 | 8 | 87 | 1 |
| 5D | 34 | 65 | 415 | 470 | 29 | 12 | 7 | 77 | 1 |
| 6A | 31 | 50 | 221 | 229 | 27 | 2 | 1 | 30 | 1 |
| 6B | 31 | 50 | 221 | 235 | 27 | 5 | 1 | 50 | 1 |
| 6C | 31 | 50 | 221 | 332 | 34 | 21 | 1 | 15 | 19 |
| 7A | 40 | 66 | 279 | 287 | 22 | 2 | 1 | 22 | 1 |
| 7B | 40 | 66 | 279 | 295 | 23 | 2 | 1 | 22 | 1 |
| 7C | 40 | 66 | 279 | 359 | 29 | 13 | 1 | 22 | 1 |
| 8A | 30 | 63 | 385 | 399 | 25 | 0 | 4 | 40 | 1 |
| 8B | 30 | 63 | 385 | 399 | 23 | 0 | 2 | 41 | 1 |
| 8C | 30 | 63 | 385 | 419 | 26 | 4 | 1 | 26 | 1 |
| 9A | 50 | 92 | 323 | 336 | 59 | 0 | 13 | 338 | 7 |
| 9B | 50 | 92 | 323 | 336 | 59 | 1 | 13 | 338 | 7 |
| 9C | 50 | 92 | 323 | 336 | 44 | 4 | 10 | 250 | 1 |
| 9D | 50 | 92 | 323 | 378 | 61 | 17 | 11 | 176 | 1 |
| 10A | 50 | 97 | 424 | 433 | 38 | 0 | 7 | 142 | 1 |
| 10B | 50 | 97 | 424 | 433 | 38 | 1 | 7 | 142 | 1 |
| 10C | 50 | 97 | 424 | 435 | 36 | 2 | 6 | 119 | 1 |
| 10D | 50 | 97 | 424 | 483 | 39 | 20 | 4 | 60 | 1 |

TABLE 4. Upper-bound results for the Benavent instances.

| Instance no. | $\|V\|$ | $\|E\|$ | UB1 | SECONDS UB1 | UB2 | SECONDS UB2 | DEVIATION % |
|---|---|---|---|---|---|---|---|
| 1A | 24 | 39 | 189 | 6 | 183 | 46 | 0.00 |
| 1B | 24 | 39 | 197 | 4 | 183 | 65 | 0.00 |
| 1C | 24 | 39 | 257 | 1 | 218 | 250 | 6.34 |
| 2A | 24 | 34 | 263 | 3 | 247 | 39 | 0.00 |
| 2B | 24 | 34 | 271 | 2 | 247 | 43 | 0.00 |
| 2C | 24 | 34 | 418 | 1 | 340 | 575 | 8.97 |
| 3A | 24 | 35 | 89 | 4 | 85 | 37 | 0.00 |
| 3B | 24 | 35 | 89 | 2 | 91 | 41 | 4.71 |
| 3C | 24 | 35 | 113 | 4 | 96 | 122 | 5.49 |
| 4A | 41 | 69 | 424 | 43 | 406 | 269 | 1.00 |
| 4B | 41 | 69 | 424 | 35 | 406 | 403 | 1.00 |
| 4C | 41 | 69 | 434 | 21 | 410 | 603 | 1.99 |
| 4D | 41 | 69 | 524 | 11 | 460 | 1537 | 7.48 |
| 5A | 34 | 65 | 471 | 32 | 456 | 243 | 5.56 |
| 5B | 34 | 65 | 479 | 23 | 455 | 467 | 5.32 |
| 5C | 34 | 65 | 491 | 18 | 434 | 857 | 0.00 |
| 5D | 34 | 65 | 589 | 9 | 512 | 1358 | 8.94 |
| 6A | 31 | 50 | 251 | 12 | 230 | 158 | 0.44 |
| 6B | 31 | 50 | 273 | 8 | 247 | 339 | 5.11 |
| 6C | 31 | 50 | 405 | 3 | 369 | 791 | 11.14 |
| 7A | 40 | 66 | 303 | 41 | 287 | 286 | 0.00 |
| 7B | 40 | 66 | 303 | 32 | 295 | 354 | 0.00 |
| 7C | 40 | 66 | 413 | 9 | 371 | 1049 | 3.34 |
| 8A | 30 | 63 | 415 | 23 | 403 | 210 | 1.00 |
| 8B | 30 | 63 | 421 | 19 | 399 | 319 | 0.00 |
| 8C | 30 | 63 | 491 | 6 | 442 | 1273 | 5.48 |
| 9A | 50 | 92 | 336 | 300 | 342 | 1126 | 0.00 |
| 9B | 50 | 92 | 336 | 176 | 337 | 658 | 0.00 |
| 9C | 50 | 92 | 342 | 108 | 352 | 648 | 1.78 |
| 9D | 50 | 92 | 379 | 51 | 413 | 2873 | 0.26 |
| 10A | 50 | 97 | 440 | 290 | 450 | 686 | 1.61 |
| 10B | 50 | 97 | 448 | 199 | 450 | 483 | 3.46 |
| 10C | 50 | 97 | 450 | 178 | 456 | 700 | 3.44 |
| 10D | 50 | 97 | 486 | 40 | 522 | 4739 | 0.62 |

TABLE 5. Lower-bound results for the Hertz–Laporte–Mittaz instances.

| No. IFs | Instance no. | $\|V\|$ | $\|E\|$ | $\|R\|$ | LB1 | LB2 | R-ODD | CAPACITY | BACKTREE | TWIN BACKTREE | NODES |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | G1.1 | 40 | 437 | 263 | 1526 | 1531 | 37 | 0 | 1 | 18 | 1 |
| | G2.1 | 40 | 429 | 241 | 1239 | 1256 | 37 | 2 | 1 | 35 | 1 |
| | G3.1 | 40 | 359 | 166 | 845 | 852 | 33 | 0 | 2 | 37 | 3 |
| | G4.1 | 40 | 359 | 205 | 1079 | 1085 | 43 | 0 | 1 | 17 | 1 |
| | G5.1 | 40 | 429 | 211 | 1111 | 1115 | 64 | 1 | 1 | 24 | 3 |
| | G6.1 | 40 | 445 | 201 | 1044 | 1047 | 38 | 1 | 1 | 33 | 1 |
| | G7.1 | 40 | 351 | 161 | 874 | 881 | 26 | 1 | 1 | 42 | 1 |
| | G8.1 | 40 | 406 | 183 | 1062 | 1104 | 27 | 4 | 1 | 19 | 1 |
| | G9.1 | 40 | 422 | 224 | 1151 | 1155 | 28 | 0 | 1 | 25 | 1 |
| | G10.1 | 40 | 406 | 208 | 1149 | 1151 | 35 | 2 | 2 | 38 | 1 |
| 2 | G1.2 | 40 | 437 | 263 | 1526 | 1528 | 35 | 0 | 1 | 17 | 1 |
| | G2.2 | 40 | 429 | 241 | 1239 | 1241 | 44 | 1 | 1 | 20 | 1 |
| | G3.2 | 40 | 359 | 166 | 845 | 851 | 29 | 0 | 2 | 29 | 3 |
| | G4.2 | 40 | 359 | 205 | 1079 | 1085 | 44 | 0 | 1 | 16 | 1 |
| | G5.2 | 40 | 429 | 211 | 1111 | 1115 | 61 | 0 | 2 | 44 | 1 |
| | G6.2 | 40 | 445 | 201 | 1044 | 1047 | 51 | 0 | 2 | 45 | 3 |
| | G7.2 | 40 | 351 | 161 | 874 | 878 | 40 | 0 | 3 | 134 | 7 |
| | G8.2 | 40 | 406 | 183 | 1062 | 1078 | 36 | 4 | 1 | 19 | 1 |
| | G9.2 | 40 | 422 | 224 | 1151 | 1155 | 43 | 0 | 2 | 70 | 5 |
| | G10.2 | 40 | 406 | 208 | 1149 | 1151 | 35 | 0 | 2 | 23 | 1 |
| 3 | G1.3 | 40 | 437 | 263 | 1526 | 1528 | 35 | 0 | 1 | 18 | 1 |
| | G2.3 | 40 | 429 | 241 | 1239 | 1240 | 39 | 0 | 1 | 38 | 1 |
| | G3.3 | 40 | 359 | 166 | 845 | 845 | 26 | 0 | 0 | 0 | 1 |
| | G4.3 | 40 | 359 | 205 | 1079 | 1085 | 42 | 0 | 2 | 45 | 5 |
| | G5.3 | 40 | 429 | 211 | 1111 | 1113 | 25 | 0 | 1 | 22 | 1 |
| | G6.3 | 40 | 445 | 201 | 1044 | 1047 | 41 | 0 | 2 | 34 | 1 |
| | G7.3 | 40 | 351 | 161 | 874 | 877 | 31 | 0 | 1 | 125 | 3 |
| | G8.3 | 40 | 406 | 183 | 1062 | 1077 | 38 | 2 | 3 | 33 | 3 |
| | G9.3 | 40 | 422 | 224 | 1151 | 1152 | 33 | 0 | 1 | 33 | 1 |
| | G10.3 | 40 | 406 | 208 | 1149 | 1152 | 35 | 0 | 1 | 22 | 1 |

TABLE 6. Upper-bound results for the Hertz–Laporte–Mittaz instances.

| No. IFs | Instance no. | $\|V\|$ | $\|E\|$ | $\|R\|$ | UB1 | SECONDS UB1 | UB2 | SECONDS UB2 | DEVIATION % |
|---|---|---|---|---|---|---|---|---|---|
| 1 | G1.1 | 40 | 437 | 263 | 1542 | 203 | 1531 | 3041 | 0.00 |
| | G2.1 | 40 | 429 | 241 | 1286 | 4 | 1271 | 12,431 | 1.19 |
| | G3.1 | 40 | 359 | 166 | 864 | 13 | 855 | 2682 | 0.35 |
| | G4.1 | 40 | 359 | 205 | 1109 | 18 | 1087 | 2861 | 0.18 |
| | G5.1 | 40 | 429 | 211 | 1129 | 8 | 1119 | 2969 | 0.35 |
| | G6.1 | 40 | 445 | 201 | 1069 | 4 | 1058 | 2543 | 1.05 |
| | G7.1 | 40 | 351 | 161 | 903 | 3 | 888 | 5054 | 0.79 |
| | G8.1 | 40 | 406 | 183 | 1170 | 1 | 1146 | 15,802 | 3.80 |
| | G9.1 | 40 | 422 | 224 | 1161 | 61 | 1157 | 1673 | 0.17 |
| | G10.1 | 40 | 406 | 208 | 1171 | 6 | 1170 | 6266 | 1.65 |
| 2 | G1.2 | 40 | 437 | 263 | 1537 | 203 | 1532 | 1482 | 0.26 |
| | G2.2 | 40 | 429 | 241 | 1265 | 5 | 1255 | 1293 | 1.13 |
| | G3.2 | 40 | 359 | 166 | 861 | 16 | 851 | 1113 | 0.00 |
| | G4.2 | 40 | 359 | 205 | 1089 | 20 | 1089 | 1788 | 0.37 |
| | G5.2 | 40 | 429 | 211 | 1127 | 9 | 1116 | 3180 | 0.09 |
| | G6.2 | 40 | 445 | 201 | 1063 | 4 | 1047 | 8935 | 0.00 |
| | G7.2 | 40 | 351 | 161 | 894 | 3 | 881 | 7358 | 0.34 |
| | G8.2 | 40 | 406 | 183 | 1133 | 1 | 1118 | 17,508 | 3.71 |
| | G9.2 | 40 | 422 | 224 | 1158 | 63 | 1160 | 4619 | 0.26 |
| | G10.2 | 40 | 406 | 208 | 1161 | 6 | 1156 | 9438 | 0.43 |
| 3 | G1.3 | 40 | 437 | 263 | 1540 | 220 | 1534 | 1645 | 0.39 |
| | G2.3 | 40 | 429 | 241 | 1260 | 7 | 1247 | 1928 | 0.56 |
| | G3.3 | 40 | 359 | 166 | 857 | 16 | 855 | 1115 | 1.18 |
| | G4.3 | 40 | 359 | 205 | 1092 | 23 | 1091 | 1873 | 0.55 |
| | G5.3 | 40 | 429 | 211 | 1123 | 12 | 1115 | 3560 | 0.18 |
| | G6.3 | 40 | 445 | 201 | 1055 | 5 | 1054 | 7890 | 0.66 |
| | G7.3 | 40 | 351 | 161 | 889 | 14 | 882 | 6500 | 0.57 |
| | G8.3 | 40 | 406 | 183 | 1124 | 3 | 1113 | 16,543 | 3.34 |
| | G9.3 | 40 | 422 | 224 | 1154 | 78 | 1153 | 5329 | 0.09 |
| | G10.3 | 40 | 406 | 208 | 1160 | 7 | 1163 | 8263 | 0.69 |

The results presented in Tables 2 and 6 show that UB2 is almost always better than is UB1, but this comes at the expense of much larger computation times. The average percentage deviation of the best upper bound over the best lower bound is 1.25 for the DeArmon instances (maximum value: 5.70), 2.78 for the Benavent instances (maximum value: 11.14), and 0.81 for the Hertz–Laporte–Mittaz instances (maximum value: 3.80). This confirms the quality of both our lower- and upper-bounding procedures.

## 5. CONCLUSIONS

We have introduced the CARPIF, a variant of the CARP in which the vehicle may unload or replenish at IFs. Applications of the CARPIF are encountered, for example, in garbage collection and road gritting. We have investigated several properties of the CARPIF and provided lower- and upper-bounding procedures. Computational results on three sets of instances confirm the quality of both bounds.


### Acknowledgments

Thanks are due by the authors to two anonymous referees for their valuable comments.


## REFERENCES


[1] A.A. Assad and B.L. Golden, "Arc routing methods and applications," Network routing, Handbooks in operations research and management science, Vol. 8, M.O. Ball, T.L. Magnanti, C.L. Monma, and G.L. Nemhauser (Editors), North-Holland, Amsterdam, pp. 375–483.

[2] J.E. Beasley, Route first–cluster second methods for vehicle routing, Omega 11 (1983), 403–408.

[3] J.M. Belenguer and E. Benavent, The capacitated arc routing problem: Valid inequalities and facets, Comput Optim Appl 10 (1997), 165–187.

[4] J.M. Belenguer and E. Benavent, A cutting plane algorithm for the capacitated arc routing problem, Working paper, Departamento de Estadistica e Investigacion Operativa, Universidad de Valencia, 1997.

[5] E. Benavent, V. Campos, A. Corberán, and E. Mota, The capacitated arc routing problem: Lower bounds, Networks 22 (1992), 669–690.

[6] A. Corberán and J.M. Sanchis, A polyhedral approach to the rural postman problem, Eur J Oper Res 79 (1994), 95–114.

[7] CPLEX Optimization Inc., Using the CPLEX callable library and CPLEX mixed integer library, 1998.

[8] J. DeArmon, A comparison of heuristics for the capacitated Chinese postman problem, Master Thesis, University of Maryland at College Park, 1981.

[9] R.W. Eglese and L.Y.O. Li, Efficient routing for winter gritting, J Oper Res Soc 43 (1992), 1031–1034.

[10] H.A. Eiselt, M. Gendreau, and G. Laporte, Arc routing problems, Part I: The Chinese postman problem, Oper Res 43 (1995), 231–242.

[11] H.A. Eiselt, M. Gendreau, and G. Laporte, Arc routing problems, Part II: The rural postman problem, Oper Res 43 (1995), 399–414.

[12] M. Fischetti, J.J. Salazar, and P. Toth, A branch and cut algorithm for the symmetric generalized traveling salesman problem, Oper Res 45 (1997), 378–394.

[13] G.N. Frederickson, Approximation algorithms for some postman problems, SIAM J Comput 7 (1979), 178–193.

[14] G. Ghiani and G. Laporte, A branch-and-cut algorithm for the undirected rural postman problem, Math Program Ser A 87 (2000), 467–481.

[15] B.L. Golden and R.T. Wong, Capacitated arc routing problems, Networks 11 (1981), 305–315.

[16] M. Grötschel, L. Lovàsz, and A. Schrijver, Geometric algorithms and combinatorial optimization, Springer-Verlag, Berlin, 1993.

[17] A. Hertz, G. Laporte, and M. Mittaz, A tabu search heuristic for the capacitated arc routing problem, Oper Res 48 (2000), 129–135.

[18] A. Hertz, G. Laporte, and P. Nanchen-Hugo, Improvement procedures for the undirected rural postman problem, INFORMS J Comput 11 (1999), 53–62.

[19] K. Jansen, Bounds for the general capacitated routing problem, Networks 23 (1993), 165–173.

[20] M. Jünger, G. Reinelt, and G. Rinaldi, "The traveling salesman problem," Network models, Handbooks in operations research and management science, M.O. Ball, T.L. Magnanti, C.L. Momna, and G.L. Nemhauser (Editors), North-Holland, Amsterdam, 1995, pp. 225–230.

[21] P.F. Lemieux and L. Campagna, The snow plowing problem solved by a graph theory algorithm, Civil Eng Syst 1 (1984), 337–341.

[22] M.W. Padberg and M.R. Rao, Odd minimum cut-sets and b-matchings, Math Oper Res 7 (1982), 67–80.

[23] C. Wirasinghe and N.M. Waters, An approximate procedure for determining the number, capacities and locations of solid waste transfer-stations in an urban region, Eur J Oper Res 12 (1983), 105–111.